

LangGraph - Complete Study Notes for MLOps Quiz

1. What is LangGraph?

LangGraph is an **open-source library** authored by **LangChain**.

It was designed to help developers build AI workflows that are:

- **Multiactor** → multiple "workers" (LLMs, tools, agents) working together
- **Multistep** → tasks broken into many steps
- **Stateful** → remembers information as it runs
- **Cognitive architectures** → mimics how a "thinking system" works

👉 **Key idea:** LangGraph models AI workflows as a **graph** instead of a **chain**.

Analogy: Think of a chain like a single-lane road (one step → next step → next step). A graph is like a city map with many intersections — you can branch, loop back, take detours, and have multiple roads working in parallel.

2. LangGraph (Continued)

- It is a **low-level orchestration framework and runtime** for building, managing, and deploying **long-running, stateful agents**.
- Very **low-level** — focused entirely on **agent orchestration** (the "traffic control" of agents).
- You **don't need LangChain** to use LangGraph (they are independent).
- LangChain provides **prebuilt agents** for common LLM and tool-calling loops.
- **Inspirations behind LangGraph:**
 - **Pregel** and **Apache Beam** → for the underlying graph engine
 - **NetworkX** → for the public interface (how developers interact with it)
- Built by **LangChain Inc.**, but usable independently.

Analogy: LangGraph is like the engine of a car — powerful but low-level. LangChain provides the steering wheel, dashboard, and seats (higher-level abstractions). You can use the engine without the rest of the car if you want full control.

3. LangGraph - Multiactor

Core idea: A team of specialists can build something none of them could build alone.

Similarly, an LLM prompt becomes much more powerful when paired with related artifacts:

- **Prompts**
- **Tools**
- **Agents**
- **MCP Servers**, etc.

Example:

- Simple flow: Input → LLM → Output
- Multiactor flow: Input → LLM ↔ Searcher → Output (LLM can call a Searcher tool back and forth)

Analogy: A doctor (LLM) is good alone, but if paired with a lab technician (Searcher tool), nurse (memory), and pharmacist (action tool), they form a much more powerful healthcare team.

4. Actor Coordination

When you have multiple actors working together, you need a **coordination layer** to manage:

1. **Define the actors** → these become **nodes** in a graph
2. **How they hand off work** → these are the **edges** of the graph
3. **Schedule execution** → who runs when

Why a single central state is powerful:

- **Snapshot and store** the central state during/after each computation
- **Pause and resume** execution → makes error recovery easy
- **Implement human-in-the-loop controls** → a human can step in and approve/edit

Analogy: Imagine a kitchen with multiple chefs (LLMs and tools). The "central state" is the recipe board on the wall — every chef looks at it, updates it, and the head chef (coordinator) decides who cooks next. If the kitchen catches fire, you can pause, save the state of the dish, and resume from exactly where you stopped.

5. Graph in LangGraph (DAG!)

DAG = Directed Acyclic Graph (directed = arrows have direction; acyclic = no infinite loops by default).

Each graph is made of three things:

◆ State

- The **data** received from outside the application, modified and produced as the app runs.
- It's the "shared notebook" all nodes read from and write to.

◆ Nodes

- Each **step** to be taken.
- Nodes are simply **Python/JS functions** (or tools).
- They **receive the current state as input** and can return an **update to that state** (add/modify/remove data).

◆ Edges

- The **connections** between nodes.
- Determine the **path** from the first node to the last.
- Two types:
 - **Fixed edges** → "After Node B, always visit Node D"
 - **Conditional edges** → "Evaluate a function to decide what comes after Node C"

Analogy: State is the shared whiteboard. Nodes are the people writing on the whiteboard. Edges are the rules saying "after Alice writes, Bob writes next."

6. Implementing a Basic Graph

A basic graph structure looks like:

```
Start → Node 1 → Node 2 → End
           ↘ Node 3 ↗
```

So Node 1 can branch into either Node 2 or Node 3, and both lead to End.

7. Graph Building Blocks

To build any LangGraph graph, you need:

1. **Graph state** → an object holding the workflow's state
2. **Nodes** → Python functions
3. **Edges** → connections between nodes
 - **Normal Edges:** Always go from Point A → Point B

- **Conditional Edges:** Logic (a Python function) decides which path to take.
 - ⚠️ **Note: Not really an edge!** It's actually a function that picks the next node.
4. **START / END Nodes:** Special nodes marking where the graph begins and ends
 5. **Compile the graph** (and optionally **display** it to verify the structure)
-

8. Creating Our First Graph (Code)

```
python

# Build graph
builder = StateGraph(State)
builder.add_node("node_1", node_1)
builder.add_node("node_2", node_2)
builder.add_node("node_3", node_3)

# Logic
builder.add_edge(START, "node_1")
builder.add_conditional_edges("node_1", decide_mood)
builder.add_edge("node_2", END)
builder.add_edge("node_3", END)

graph = builder.compile()
display(Image(graph.get_graph().draw_mermaid_png()))
```

What happens:

- `StateGraph(State)` → create a graph that uses `State` as its data type.
 - `add_node` → register each step.
 - `add_edge(START, "node_1")` → the graph begins at node_1.
 - `add_conditional_edges("node_1", decide_mood)` → after node_1, run the `decide_mood` function to choose node_2 or node_3.
 - `compile()` → finalize the graph so it can run.
 - `draw_mermaid_png()` → visualize it as an image.
-

9. Conditional Edge (Code Example)

```
python
```

```
def decide_mood(state) -> Literal["node_2", "node_3"]:
    # Often, we will use state to decide on the next node to visit
    user_input = state['graph_state']

    # Here, let's just do a 50 / 50 split between nodes 2, 3
    if random.random() < 0.5:
        # 50% of the time, we return Node 2
        return "node_2"

    # 50% of the time, we return Node 3
    return "node_3"
```

Explanation:

- The function reads the **state**.
- Decides (randomly here, but typically based on logic) whether to go to **node_2** or **node_3**.
- The return value (a string) **must match** a node name.

Analogy: It's like a traffic cop at an intersection. The cop looks at conditions (state) and waves the car (flow) toward one of several roads (nodes).

10. Reducers

Reducers define how simultaneous/competing updates from multiple actors are combined.

Scenario:

Imagine a graph with **two parallel branches**:

- Branch A produces `{"messages": [msgA]}`
- Branch B produces `{"messages": [msgB]}`

Both want to update the same key `messages`. But:

- Should the final state contain only msgA?
- Only msgB?
- Both?
- In what order?

A reducer answers that question deterministically.

Key facts:

- Typically defined in the **state schema** (the typed definition of your state).

- You're basically defining **rules** for combining multiple updates.
- **Reducers are crucial for parallelism.**

Analogy: Imagine two employees both updating the same Excel cell at the same time. Without a rule, you'd lose data or get confused. A reducer is the rule that says: "When two updates collide, append both" or "keep only the latest" or "sum them up."

11. Reducer Patterns

Goal	Reducer Pattern
Keep all values	List append / set union
Keep only one authoritative value	Last write wins
Combine partial structured data	Dict merge
Aggregate metrics	Sum / Max / Min

Example: Message Accumulation

LangGraph lets you define the `messages` state key with a reducer that **concatenates lists**:

```
new_messages = old_messages + node1_messages + node2_messages
```

This is done using the built-in `add_messages` reducer.

12. Message Reducer

```
python

from typing import Annotated
from langgraph.graph.message import add_messages

class MessagesState(TypedDict):
    messages: Annotated[list[AnyMessage], add_messages]
```

Type	Meaning
<code>messages</code>	The key name in the state dictionary
<code>list[AnyMessage]</code>	The type — a list of LangChain message objects (Human, AI, Tool, etc.)
<code>add_messages</code>	The reducer function — tells LangGraph how to update this key
<code>Annotated[... , ...]</code>	From Python's <code>typing</code> module. Lets you attach extra metadata to a type hint without changing the type itself. LangGraph reads that metadata at graph construction time and uses <code>add_messages</code> as the merge strategy.

Important difference:

- **Without reducer:** `messages` becomes `[new_message]` — previous messages are **lost**.
- **With `add_messages` reducer:** `new_message` is **appended** — conversation history is preserved! ✓

Analogy: Without the reducer, every new sticky note replaces the old ones on the fridge. With `add_messages`, every new sticky note gets added to a growing collection.

13. Reducer for Messages: MessagesState

- `MessagesState` is a **pre-built** conversation-oriented state schema.
- It eliminates boilerplate when your graph state is basically "a growing list of messages."
- Under the hood, it uses the `add_messages` **reducer**, which merges two lists of messages, **updating existing messages by ID**.
- Makes the state **append-only by default**.

⚠ But `add_messages` is NOT just append!

Besides merging two lists, it also **updates existing messages** if a new message is provided with an **existing ID**.

```
python
msgs1 = [HumanMessage(content="Hello", id="1")]
msgs2 = [HumanMessage(content="Hello again", id="1")]
add_messages(msgs1, msgs2)
# Result: [HumanMessage(content='Hello again', id='1')]
```

Analogy: It's like editing a Slack message — if the ID matches, the new content replaces the old one. If the ID is new, the message is added.

14. LangGraph Helpers — `tools_condition`

`tools_condition` is a **prebuilt routing (conditional-edge) function** used in tool-calling agent loops (ReAct-style).

Its job:

1. Look at the **last model (AI) message** in your state.
2. **If it contains** `tool_calls` → route to the **tools** execution node.
3. **Otherwise** → end the graph.

```
python
```

```
tools_condition(state, messages_key="messages")  
    → Literal["tools", "__end__"]
```

- Returns `"tools"` when the last message is an `AIMessage` and has one or more `tool_calls`.
- Returns `"__end__"` when there are no tool calls (the agent is ready to respond, workflow terminates).

Analogy: A receptionist at a hotel. If the guest needs something done (tool call), they send the request to housekeeping (tools node). If the guest is satisfied, they let them leave (end graph).

15. Agents in LangGraph (LLM Tool Workflow)

Structure:

- **Assistant Node** → contains our LLM with Tools.
- Based on the LLM's response:
 - If there's a **tool call** → tool is invoked, response is sent to a **conditional edge**.
 - Conditional edge contains `tools_condition`.
 - If response is `"tools"` → flow goes back to the **assistant node**.
 - If not → graph ends!

```
START → assistant ⇌ tools  
      ↓  
    END
```

16. Subgraphs

A **subgraph** is simply **another graph used as a node inside a larger LangGraph**.

Why subgraphs work:

- In LangGraph, nodes are just **functions or components** that take state, perform work, and return updated state.
- Since nodes can be **arbitrary functions** (not just LLM calls), a **compiled graph can also act as a node** in another graph.
- This creates a **hierarchical workflow**.

Why use them?

- **Distributing development:** Different teams can work on different parts of the graph independently.

Analogy: A subgraph is like a "function within a function" or a department within a company. The marketing department (subgraph) has its own internal workflow, but from the CEO's view (main graph), it's just one step in the whole company's process.

17. Subgraph Patterns

Pattern	When to use	State schemas
Call a subgraph inside a node	Parent and subgraph have different state schemas (no shared keys), or you need to transform state between them	You write a wrapper function that maps parent state → subgraph input, and subgraph output → parent state
Add a subgraph as a node	Parent and subgraph share state keys — subgraph reads from and writes to the same channels as the parent	You pass the compiled subgraph directly to <code>add_node</code>

18. Invoking Subgraph Inside a Node

Use this when **parent graph and subgraph have different state schemas (no shared keys)**.

This is common when you want to keep a **private message history for each agent** in a multi-agent system.

The node function:

1. **Transforms parent state → subgraph state**
2. **Invokes the subgraph**
3. **Transforms results back → parent state**

```
python

class SubgraphState(TypedDict):
    bar: str

# Subgraph
def subgraph_node_1(state: SubgraphState):
    return {"bar": "hi! " + state["bar"]}

# Parent graph
class State(TypedDict):
    foo: str

def call_subgraph(state: State):
    # Transform parent state → subgraph state
    subgraph_output = subgraph.invoke(
        {"bar": state["foo"]})
    # Transform response → parent state
    return {"foo": subgraph_output["bar"]}
```

Analogy: A translator at the United Nations. The English speaker's words (parent state) need to be translated into French (subgraph state). The French response is then translated back into English (parent state).

19. Add Subgraph as Node

Use this when **parent and subgraph share state keys**.

- You can pass a **compiled subgraph directly** to `add_node`.
- The subgraph **reads from and writes to the parent's state channels automatically**.
- Common in **multi-agent systems** where agents communicate over a shared `messages` key.

Example:

```
python
```

```

sub_builder = StateGraph(State)

def node1(state): ...
def node2(state): ...

sub_builder.add_node("node1", node1)
sub_builder.add_node("node2", node2)
sub_builder.add_edge(START, "node1")
sub_builder.add_edge("node1", "node2")
subgraph = sub_builder.compile()

main_builder = StateGraph(State)
main_builder.add_node("analysis", subgraph) # Subgraph plugged in directly
main_builder.add_edge(START, "analysis")
main_graph = main_builder.compile()

```

Analogy: Two teams in the same company using the same shared Google Doc. They don't need translators because they speak the same "language" (state schema).

20. LangGraph - Persistence

- LangGraph has a built-in **persistence layer** that saves graph state as **checkpoints**.
- When you compile a graph with a **checkpointer**, a snapshot of the graph state is saved at **every step of execution**, organized into **threads**.

Key Terms:

Concept	Meaning
Graph	Control flow of nodes, edges
Super-steps	Each sequential node is a separate super-step; parallel nodes share the same super-step
Checkpoints	State + relevant metadata packaged at every super-step
Thread	Collection of checkpoints
StateSnapshot	The Python type used for checkpoints

Analogy: Like saving your progress in a video game. Every checkpoint = a save file. A thread is your "campaign" containing all save files for one playthrough.

Durable execution = a technique where a process **saves its progress at key points**, allowing it to **pause and resume exactly where it left off**.

How to enable:

1. **Specify a checkpoint** to save workflow progress.
2. **Specify a thread identifier** when running the workflow → tracks history for that workflow instance.
3. **Wrap non-deterministic operations or operations with side effects** (random number generation, file writes, API calls) inside **tasks** so they aren't repeated on resume.

Analogy: Like a nuclear power plant's emergency shutdown — when restarted, it doesn't re-detonate the same reactions; it picks up exactly where it left off without redoing irreversible steps.

22. Tasks (Code Example)

```
python

def call_api(state: State):
    """Example node that makes an API request."""
    result = requests.get(state['url']).text[:100]
    return {"result": result}

@task
def _make_request(url: str):
    """Make a request."""
    return requests.get(url).text[:100]
```

The `@task` decorator marks the function so that, on resume, it won't be re-executed if it already ran successfully.

23. LangSmith Deployment

LangSmith Deployment is a **workflow orchestration runtime** purpose-built for **agent workloads**.

What it does:

- Takes your agent code (e.g., a LangGraph graph) and **exposes it as a production-ready API service**.
- Built-in features:

- **State management**
 - **Scaling**
 - **Monitoring**
 - **Persistence**
- **Framework-agnostic** → can deploy agents from LangGraph or other frameworks.

LangSmith's deployment layer provides:

- **Durable execution** of agent workloads with persistent state & conversation memory
- **API endpoints** for interacting with agents
- **Monitoring and debugging tools**
- **Deployment infrastructure** (cloud, hybrid, or self-hosted)

Analogy: LangSmith Deployment is like AWS for agents — you give it your code, it runs it at scale, monitors it, scales it, and gives you APIs to talk to it.

24. Agent Server

- When you deploy to LangSmith, you're basically deploying to an **Agent Server** — a backend service designed to **execute agent graphs**.
- It exposes a structured API for managing agents and their executions.

Agent Server API is built around 4 main concepts:

Entity	What it represents
Assistant	A configured agent
Thread	An execution context with shared state
Run	One execution of the agent
Store	Persistent key-value memory

Analogy: Think of a call center.

- **Assistant** = a specific call agent (with their script and tools).
- **Thread** = an ongoing conversation with one customer.
- **Run** = a single phone call.
- **Store** = the CRM database that remembers customers across calls.

25. Agent Server: Assistant

- **Assistants** are an Agent Server concept that allow you to **manage configurations** (e.g., prompts, LLM selection, tools) **separately from your graph's core logic**.
- You can create **multiple specialized versions** of the same graph architecture with **different behavior at runtime**.

Example:

Same graph (ReAct Agent), but two different Assistants:

- **Assistant #1: Blog writing agent** → Model: Anthropic, Prompt: "write a long blog..."
- **Assistant #2: Tweet writing agent** → Model: OpenAI, Prompt: "write a concise tweet..."

Each Assistant can also have **versions** (v1, v2, v3, ...).

Analogy: Same car model, different trims. The chassis (graph) is identical, but one is configured as a luxury edition and another as a sport edition.

26. Assistant (Code Example)

An assistant is just an instance of a graph with a specific configuration. Multiple assistants can reference the **same graph** but contain **different configurations** (prompts, models, tools).

```
python

assistant = await client.assistants.create(
    "writing_agent",
    context={"model_name": "openai"},
    name="Blog Writer"
)

result = await client.runs.wait(
    thread["thread_id"],
    assistant["assistant_id"],
    input={
        "messages": [...]
    },
)
```

27. LangGraph Store

- The **Store** is the **long-term memory layer**.
- A persistent **key-{value|document}** store that **survives across threads and conversations**.
- Unlike thread state/checkpoints, which are scoped to runs or threads, the **Store outlives them**.
- Agent Server stores long-term memory in **PostgreSQL** by default; can use a custom store implementation.
- When you deploy on Agent Server, **the server injects the configured store into graph execution** — completely **decoupled from the code**.
- ⚠️ **Do NOT configure the store in your graph code for Agent Server** — the server manages it for runtime operations.

Analogy: Threads/checkpoints = your short-term memory (today's events). The Store = your long-term memory (childhood memories, learned skills) that persists forever.

28. LangGraph Store (Code Example)

```
python
```

```
# Writing to store
namespace = ["users", "user123"]
key = "profile"

# Put / upsert a document
await client.store.put_item(
    namespace, key,
    {
        "name": "Ahmed",
        "preferences": {"tone": "concise", "language": "en"}
    }
)

# Read it back
item = await client.store.get_item(namespace, key)

# Search under a namespace prefix
results = await client.store.search_items(
    ["users"],
    filter={"preferences.language": "en"},
    query="concise tone"
)

# List namespaces
namespaces = await client.store.list_namespaces()
```

29. LangGraph Store — Multi-Tenancy Pattern

- Long-term memory is stored as **JSON documents** under a **namespace + key** model.
- Search works **within namespace prefixes** with filters and **semantic ranking**.
- ☒ **Best practice:** include **tenant ID in the namespace** so memories don't leak across users.

```
python

("tenant_a", "customers", "c42")
```

This gives you:

- **Data isolation**
- **Cleaner search boundaries**
- **Easier multi-tenant enforcement**

Analogy: Like having separate filing cabinets for each company in a co-working space. No one can accidentally peek into another company's files.

30. langgraph.json

- `langgraph.json` is the **configuration file** that defines how a LangGraph application should be **built and run** by the LangGraph CLI / Agent Server.
- It tells the runtime:
 - Which graphs exist
 - How to load them
 - What dependencies to install
 - How to configure the server and runtime
- At runtime, the LangGraph CLI **starts an Agent Server** that runs graphs.
- This server **exposes API endpoints** for runs, threads, assistants, etc.
- Requires a configuration file (`langgraph.json`) describing graphs, dependencies, and environment variables.

Example `langgraph.json`:

```
json
{
  "dependencies": [".", "langchain_openai"],
  "graphs": {
    "research": "./my_agent/research.py:graph",
    "sql": "./my_agent/sql.py:graph",
    "writer": "./my_agent/writer.py:graph",
    "supervisor": "./my_agent/supervisor.py:graph"
  },
  "env": ".env"
}
```

Analogy: Like a `package.json` in Node.js or `requirements.txt` in Python — it tells the system what to install and how to start the project.

31. Dependencies

- `dependencies` tells the LangGraph CLI / Agent Server **what packages or local code it needs to install** so your graph can import and run successfully.

- When you run commands like `langgraph dev`, `langgraph build`, or `langgraph up`, the CLI reads `langgraph.json` and uses `dependencies` to **prepare the runtime environment** for your graph server.
- Determines **what Python packages and local project code** should be installed inside the server environment **before your graph is loaded**.

Valid dependency entries:

Entry	Meaning
<code>"."</code> (a single period)	Tells the CLI to look for and install the current local Python package/project
<code>"./"</code> or <code>"./my_package"</code>	A local directory path containing your <code>pyproject.toml</code> , <code>setup.py</code> , or <code>requirements.txt</code> . Point to the directory , not the file.
<code>"langchain_openai"</code> (a name)	Installs a package from the Python environment / package index so your graph can import it.

32. LangGraph CLI

Command	What it does
<code>langgraph dev</code>	Starts a lightweight local dev server , ideal for rapid testing.
<code>langgraph build</code>	Builds a Docker image of your LangGraph API server for deployment.
<code>langgraph deploy</code>	Builds and deploys a LangGraph image directly to LangSmith Deployments in a single step.
<code>langgraph dockerfile</code>	Emits a Dockerfile derived from your config for custom builds .
<code>langgraph up</code>	Starts the LangGraph API server locally in Docker . Requires Docker running, LangSmith API key for local dev, license for production.

Analogy:

- `dev` = run locally for quick testing (like `npm run dev`)
- `build` = package it up
- `deploy` = ship it to production
- `dockerfile` = give me the recipe so I can customize

12. Why is a **conditional edge** described as "not really an edge"?
 13. What are the **START** and **END** nodes?
 14. What is a **reducer** in LangGraph and why is it crucial for parallelism?
 15. Name **four reducer patterns** and the goal each one achieves.
 16. What is the difference between behavior **with** and **without** the `add_messages` reducer on a `messages` key?
 17. Why is `add_messages` "**not just append**"? Give the second behavior it provides.
 18. What is `MessagesState` and what problem does it solve?
 19. What does `tools_condition` do? When does it return `"tools"` vs `"__end__"`?
 20. Describe the structure of an **Agent in LangGraph** (assistant node, tools, conditional edge).
 21. What is a **subgraph**? Why is this even possible in LangGraph?
 22. Name the **two subgraph patterns** and when to use each.
 23. What is **persistence** in LangGraph and what is a **checkpoint**?
 24. Define **super-step**, **checkpoint**, **thread**, and **StateSnapshot**.
 25. What is **durable execution**? List the three steps to enable it.
 26. Why must non-deterministic operations be wrapped in `@task`?
 27. What is **LangSmith Deployment**? Is it tied only to LangGraph?
 28. List the **four entities** that the Agent Server API is built around.
 29. What is an **Assistant**? How is it different from the underlying graph?
 30. What is the **LangGraph Store**, and how is it different from threads/checkpoints?
 31. Why should you **NOT configure the store in your graph code** when using Agent Server?
 32. Why is including a **tenant ID in the namespace** a best practice?
 33. What is `langgraph.json` and what four things does it tell the runtime?
 34. What does the `dependencies` field in `langgraph.json` do?
 35. Differentiate the five LangGraph CLI commands: `dev`, `build`, `deploy`, `dockerfile`, `up`.
-

Technical Questions

1. Write the code to **build a basic graph** with three nodes, where node_1 conditionally routes to node_2 or node_3, and both end at END.
2. Write a **conditional edge function** `decide_mood` that returns `"node_2"` or `"node_3"` based on a 50/50 random split. Include the correct return type hint.
3. Write the code that defines a `MessagesState` class using `Annotated`, `add_messages`, and `list[AnyMessage]`.
4. What will be the output of the following?

python

```
msgs1 = [HumanMessage(content="Hello", id="1")]
msgs2 = [HumanMessage(content="Hello again", id="1")]
add_messages(msgs1, msgs2)
```

5. Write the code to **invoke a subgraph inside a node** when parent state has key `foo` and subgraph state has key `bar`.
 6. Write the code to **add a compiled subgraph as a node** in a parent graph (when both share the same state schema).
 7. Write a `@task`-decorated function `_make_request(url)` that fetches a URL and returns the first 100 characters.
 8. Write the code to **create an assistant** named "Blog Writer" using a graph called "writing_agent" with `model_name = "openai"`.
 9. Write code to **put** a profile document for `user123` into the LangGraph store under namespace `["users", "user123"]`.
 10. Write code to **search the store** for items under `["users"]` where `preferences.language == "en"`, with a query `"concise tone"`.
 11. Write a sample `langgraph.json` containing 4 graphs: research, sql, writer, supervisor — with dependencies on the local project and `langchain_openai`, and an `.env` file.
 12. What CLI command would you run to: (a) start a local dev server? (b) build a Docker image? (c) deploy to LangSmith?
-

Complex / Conceptual Questions

1. Suppose you have **two parallel branches** updating the same key `messages`. Without a reducer, what is the problem? With `add_messages`, how is it solved?
2. Explain the **trade-off** between using a subgraph **inside a node** vs. **as a node**. Give one situation where each is preferred.
3. Imagine you build a multi-agent system where **each agent has its own private message history**, but they all report into a shared coordinator. Which subgraph pattern would you use, and why?
4. Why are **reducers important for parallelism** but not strictly necessary for purely sequential graphs?
5. A graph crashes mid-execution after calling an external paid API. Explain how **durable execution** + `@task` prevents you from being charged twice when the workflow resumes.

6. Compare **threads** (short-term) vs **Store** (long-term) in LangGraph. Which would you use for: (a) chat history of a single conversation? (b) a user's long-term preferences across all conversations?
7. Why is it a problem to deploy 10 agents that all hardcode a Postgres connection in their code, vs. relying on the **Agent Server's injected store**?
8. You want to deploy the **same ReAct graph** but expose it as both a **Blog Writer (Anthropic)** and a **Tweet Writer (OpenAI)**. Explain how the **Assistant** abstraction lets you do this without duplicating graph code.
9. Why does LangGraph use a **graph (DAG)** rather than a **linear chain** for agent orchestration? Give two real benefits.
10. A team mate writes:

```
python
```

```
class State(TypedDict):  
    messages: list[AnyMessage]    # no Annotated, no reducer
```

They wonder why their conversation history keeps getting wiped out. Explain what's wrong and how to fix it.

11. Walk through what happens **end-to-end** when you run `langgraph dev`: - what file is read, - what is started, - what URLs become available, - and why this should NOT be used in production.
12. Explain why **the conditional edge function returns a string that must match a node name**, and what happens conceptually under the hood when LangGraph receives this string.
13. Why is `tools_condition` called a **"routing function"** rather than just a regular conditional? How does it integrate with the assistant-tools loop in a ReAct agent?
14. Explain how a **checkpoint** + **thread ID** + **Store** together enable a chatbot that: - remembers the current conversation, - can be paused and resumed, - and remembers the user across many separate conversations.
15. Critique the following design: a developer puts **all logic — research, SQL, writing, and supervision — into one giant graph file** with 50 nodes. How would refactoring with **subgraphs** improve this? What organizational benefit does this give a team?

✅ **Good luck with your quiz!** Review the analogies if any concept feels abstract — they're designed to anchor the technical terms in everyday intuition.